Extracting Interest Points and Their Descriptors: SIFT Features

Dr. Khalil Yousef

Computer Science and Engineering Department
Qatar University (QU)

Reading

- **Szeliski** (2nd edition): **7.1.1 -7.1.3**

Introduction

- Our goal in this topic is to first introduce you to the notion of **scale space** that plays an important role in the more modern operators like SIFT (Scale-invariant feature transform) for extracting interest points that are significantly invariant to scale, orientation and illumination changes.
- Get introduced to Harris corner detector.
- Get introduced to features matching and establishing Point-to-Point Correspondences between the images.

Recall: Gradient and Hessian Functions or Operators (Continuous Case)

- Given function: $f(x,y) = x^2 + 2xy + 3y^2$.
- Gradient: The gradient of a function is a vector containing the partial derivatives of the function with respect to each variable.
  - Partial derivatives:
    - $\partial f/\partial x = 2x + 2y$ and
    - $\partial f/\partial y = 2x + 6y$
  - Gradient vector: $\nabla f(x,y) = [\partial f/\partial x, \partial f/\partial y] = [2x + 2y, 2x + 6y]$
- Hessian Matrix: The Hessian matrix contains the second-order partial derivatives of the function (it is always symmetric).
  - Second-order partial derivatives:
    - $\partial^2 f/\partial x^2 = 2$
    - $\partial^2 f/\partial y^2 = 6$
    - $\partial^2 f/\partial x \partial y = 2$ since $\partial^2 f/\partial y \partial x = 2$
    - Hessian matrix: $H(x,y) = \begin{bmatrix} 2 & 2 \\ 2 & 6 \end{bmatrix}$

Recall: The Laplacian operator

- The Laplacian operator is a differential operator that measures the rate of change of a function's gradient.
- In simpler terms, it helps to identify areas where a function is changing rapidly or slowly.
- For a function $f(x, y)$ in two dimensions, the Laplacian is defined as:
  - $\nabla^2 f = \partial^2 f / \partial x^2 + \partial^2 f / \partial y^2$
    - $\partial^2 f / \partial x^2$ is the second partial derivative of f with respect to x
    - $\partial^2 f / \partial y^2$ is the second partial derivative of f with respect to y

Recall: An Important Fact About Images

- **An image** is a **discrete** 2D signal (NOT CONTINUOUS)

Recall: Edge Detection with the Sobel Operator

- **Edge detection** is fundamentally about detecting sudden changes in the gray levels. The simplest way to do so is to calculate at each pixel the gradient vector of the gray levels:
  - $\nabla f(x,y) = \hat{x}\frac{\partial f}{\partial x} + \hat{y}\frac{\partial f}{\partial y}$
  - The magnitude (the norm) of the gradient $\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$, gives us the strength of the edge at that point and $\theta = tan^{-1}\left(\frac{\partial f}{\partial y}/\frac{\partial f}{\partial x}\right)$ the direction of the normal to the edge.
  - The direction of the gradient is always along the max rate of change.
- Different digital approximations to the calculation of the gradient give us different edge detection operators, the most popular being the Sobel Operator.

Recall: Edge Detection with Sobel Operator

- For Sobel, you implement the $\frac{\partial}{\partial x}$ and $\frac{\partial}{\partial y}$ calculations by **convolving** an image pixel array with the 3×3 masks $\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$ and $\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$, respectively .
- Sobel Operator **Example**:
  - See https://en.wikipedia.org/wiki/Sobel_operator
  - See https://docs.opencv.org/3.4/d2/d2c/tutorial_sobel_derivatives.html

Edge Detection With the LoG Operator

- Another well-known digital approximation to the calculation of the gradient is the **LoG** edge detection operator
- **LoG** stands for Laplacian-of-Gaussian.
- Every edge operator must include some smoothing to counteract the noise enhancing property of derivative calculations.
  - [Can you see what part of Sobel amounts to smoothing?]
- For LoG, the smoothing consists of convolving the image $f(x,y)$ with a Gaussian function $g(x,y)$. That is we compute

  $\int\int f(x',y')g(x-x',y-y')dx'dy'$ where $g(x,y) = \frac{1}{2\pi\sigma^2}e^{-\frac{x^2+y^2}{2\sigma^2}}$.
- And, the derivative operation consists of applying the isotropic Laplacian operator $\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$ to the smoothed data. The edges correspond to the zero-crossing of the Laplacian operator output.

Aside: Smoothing in Sobel Operator

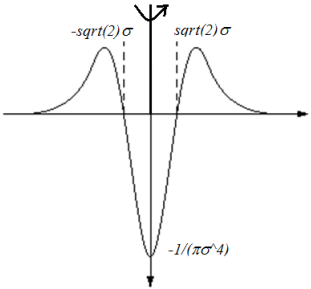
$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$$

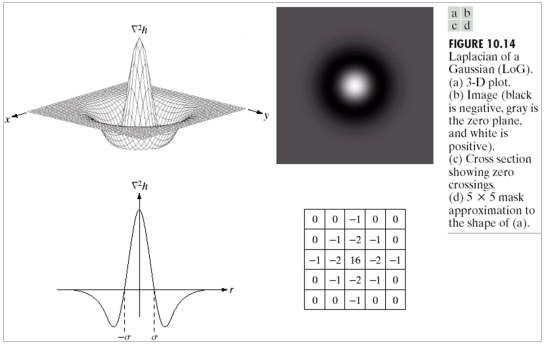
$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

Source: https://en.wikipedia.org/wiki/Sobel_operator

Edge Detection With the LoG Operator

- In practice, we combine the smoothing and the derivative operations into a single operator $h(x,y) = \left\{ \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} \right\} g(x,y) = \frac{-1}{2\pi\sigma^4} \left(2 - \frac{x^2+y^2}{\sigma^2} \right) e^{-\frac{x^2+y^2}{2\sigma^2}}$.



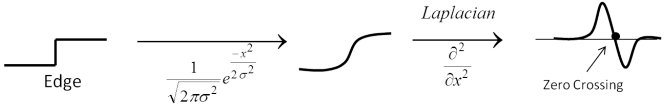


0	0	−1	0	0
0	−1	−2	−1	0
−1	−2	16	−2	−1
0	−1	−2	−1	0
0	0	−1	0	0

a b
c d
FIGURE 10.14 Laplacian of a Gaussian (LoG). (a) 3-D plot. (b) Image (black is negative, gray is the zero plane, and white is positive). (c) Cross section showing zero crossings. (d) 5×5 mask approximation to the shape of (a).

Edge Detection With the LoG Operator

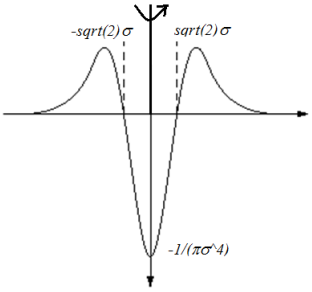
- LoG in 1-D:



- The digital approximation to the LoG is expressed as $h(m,n) = A\left(1 - k\frac{m^2+n^2}{\sigma^2}\right) e^{-\frac{m^2+n^2}{2\sigma^2}}$ with:
  - A is some amplitude,
  - k is some constant
  - $-N < m, n < N$,
  - giving us a $(2N+1) \times (2N+1)$ operator.

Edge Detection With the LoG Operator

- Digital Laplacian with no smoothing $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$.
  - As you know, σ controls the extent of smoothing.
  - To set σ, we assume a unit sampling interval between the pixels. As shown in the figure below, the radius of the control lobe in the xy-plane is $\sqrt{2}\sigma$.



Edge Detection With the LoG Operator: Rules of thumb for digital approximation of the LoG operator

- So if we set $\sigma = \sqrt{2}$, the radius will be roughly 2 pixels.
- Even though the analytical $h(x, y)$ extends over the entire xy-plane, we truncate $h(m, n)$ at 3 times the radius, that is at $3\sqrt{2}\sigma$.
  - So when $\sigma = \sqrt{2}$, we get $N = 6$, which results in a 13×13 operator. Next, we set k so that the normalization condition $\sum_{m,n=-N}^{N} h(m, n) = 0$ is satisfied.
  - Finally, we set A so that the integer quantization of the values of $h(m, n)$ retain the essential properties of the operator.
  - Note, when $\sigma = \sqrt{2}$, $|h(0,0)| = \frac{1}{\pi\sigma^4} = 0.08$.
  - So a straightforward integer quantization would cause all of the operator to disappear.
  - We make A large enough so that the farthest values of h are close to zero.

Edge Detection With the LoG Operator: Example

- See https://en.wikipedia.org/wiki/Difference_of_Gaussians
- See https://docs.opencv.org/3.4/d5/db5/tutorial_laplace_operator.html

Scale Space

Introduction - Scale Space

- Summary: From the previous slides about the LoG operator for edge detection.
  - What the **LoG** operator does **amounts to**: first smoothing the image with a Gaussian function $g(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$ and then applying the Laplacian $\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$ to the smoothed image.
  - Obviously, the larger the value of σ, the greater the smoothing effect, and the greater the suppression of small and fast changes in image gray levels. In other words, the larger the value of σ, the larger the **scale** at which the gray levels must change in order to be detectable by the LoG operator.

Introduction - Scale Space

- When any form of image representation depend on σ, the set of the representations that can be calculated for all possible values of σ constitutes **the scale space** of the image.
- Here is an important result from the scale-space theory for images: For a given image $f(x,y)$, let $ff(x,y,\sigma)$ represent its σ- smoothed version. That is, $ff(x,y,\sigma) = \int\int_{-\infty}^{\infty} f(x',y')g(x-x',y-y')dx'dy'$ where $g(x,y)$ is the Gaussian that was shown previously. The important results is $\frac{\partial}{\partial\sigma}ff(x,y,\sigma) = \sigma\nabla^2 ff(x,y,\sigma)$. Since $\nabla^2 ff(x,y,\sigma)$ is the LoG of $f(x,y)$, we can write $LoG(f(x,y)) = \frac{\partial}{\partial\sigma}ff(x,y,\sigma)$

Introduction - Scale Space

- The result in the previous slide implies that the LoG of an image $f(x, y)$ can be **approximated** by subtracting the $(\sigma + \delta\sigma)$-smoothed version of $f(x, y)$ from the σ-smoothed version. The difference of two Gaussian-smoothed versions of $f(x, y)$ for two different values of σ is referred to as the Difference - of - Gaussian (DoG).
- The fact that LoG can be approximated by DoG gives you a clue to the power of scale-space analysis.

DoF and Laplacian

- Functions for determining scale $f = \text{Kernel} * \text{Image}$

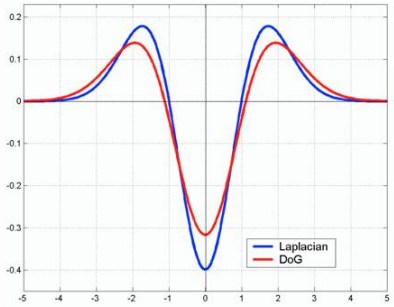
Kernels:

$$\nabla^2 g = \frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2}$$

(Laplacian)

$$DoG = G(x, y, k\sigma) - G(x, y, \sigma)$$

(Difference of Gaussians)



where Gaussian

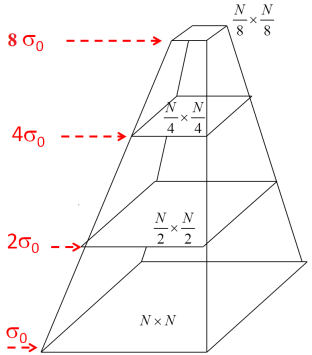
$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Note: The LoG and DoG operators are both rotation equivariant

Difference - of - Gaussian (DoG)

- The fact that we can approximate LoG$[f(x,y)]$ by DoG$[f(x,y)]$, while giving us a window into the power of scale-space analysis, has some pretty amazing computational consequences: It is much faster to calculate DoG with σ_1 and σ_2 for the two Gaussian smoothings than to calculate LoG for any given σ.
- As for the reason, since the Gaussian $g(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$ is separable in x and y, **each** of the 2-D smoothings for DoG can be carried out by two applications of 1-D smoothings, first along x and then the result along y. This cannot be done for the LoG operator $h(x,y)$ since it is not separable.
  - Recall: $h(x,y) = \left\{ \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} \right\} g(x,y) = \frac{-1}{2\pi\sigma^4} \left(2 - \frac{x^2+y^2}{\sigma^2} \right) e^{-\frac{x^2+y^2}{2\sigma^2}}$

Image Pyramids for Scale-Space Operators

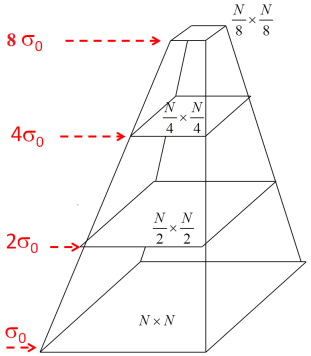


- Now that you know what a scale space is, let's talk about different kinds of pyramidal representations for an image. Pyramidal representations are at the heart of operators such as SIFT.

Image Pyramids for Scale-Space Operators

- The simplest such representations is the Gaussian Pyramid.
  - One generates a Gaussian pyramid for an image by smoothing it with successively larger Gaussian functions (meaning, Gaussians with larger σ) and arranging the sequence of smoothed images in the form of a stack that looks like a pyramid (for reasons that will be clear shortly).
  - Let's say that at the bottom of the pyramid:
    - The first application of the Gaussian is with $\sigma = \sigma_0$.
    - Subsequently, as you go from one level to the next, you double the size of σ.
    - Additionally, after smoothing you downsample the resulting image by a factor of 2.

Gaussian Pyramid



- An important practical reason for down-sampling is that you can use the same $M \times M$ digital approximation to the Gaussian for all level-to-level transitions in the pyramid.
  - The factor-of-2 down-sampling can be achieved by retaining every other row and column in the image.

Gaussian Pyramid

- The fact that each level of the pyramid is characterized by a σ that is twice the value used in the previous level can also be expressed by saying that each level of the Gaussian pyramid is one octave above the level below.
  - [In music, if two notes are one octave apart, the pitch at the higher note is twice the pitch at the lower note.]

The Laplacian (or the DoG) Pyramid

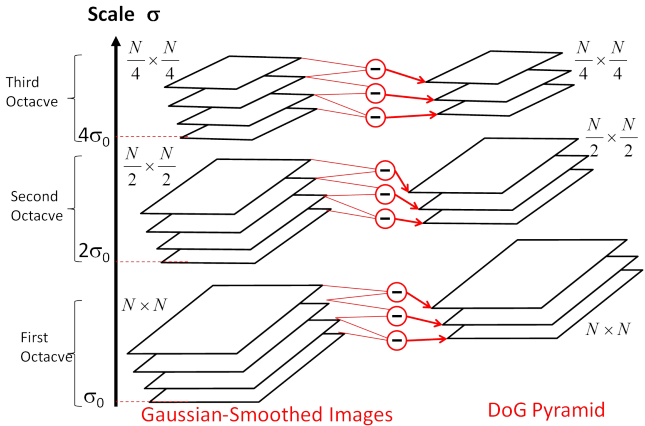


Figure: DoG Pyramid

DoG Pyramid

- The smoothed images that reside at the different levels of a Gaussian pyramid are digital versions of $ff(x,y,\sigma)$ we defined earlier for $\sigma = \sigma_0, 2\sigma_0, 4\sigma_0, 8\sigma_0, \ldots$
- Recall that a direct consequence of $\frac{\partial}{\partial\sigma} ff(x,y,\sigma) = \sigma\nabla^2 ff(x,y,\sigma)$ is that:
  - We can calculate the Laplacian of the image at scale σ by the DoG $ff(x,y,\sigma_1) - ff(x,y,\sigma_2)$ for two closely spaced scales σ_1 and σ_2.
  - **The Gaussian pyramid does not lend itself directly to the calculation of the Laplacians in this manner**
    - because the one-octave jump between two successive levels is much too large for the approximation we need for $\frac{\partial}{\partial\sigma} ff(x,y,\sigma)$.

DoG Pyramid

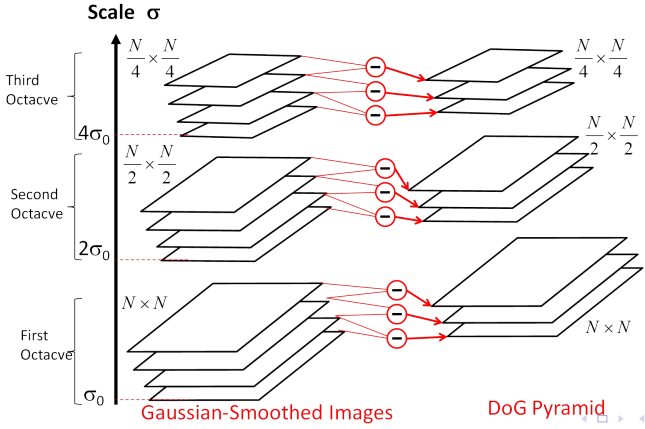
- Therefore, for calculating the Laplacians through DoG on the pyramid, **we must first calculate the Gaussian-smoothed images at additional scale values withing each octave**.
- Since the sampling points along σ constitute a geometric progression (the different levels of the Gaussian pyramid are for scale values $\sigma.2^i, i = 1,2,3,...$), which constitutes a geometric series, we would like to maintain the same form for the additional within-octave sampling of the scale space.
  - This we can do by **defining a within-octave sampling** of the scale space.
    - $\Rightarrow$ Let the within-octave scale sampling interval be $\triangle\sigma = \frac{\sigma_b}{2^s}$, where σ_b is the value of σ at the base of the octave and where s is a user-specified integer.
    - We now place the additional sampling points at $\triangle\sigma.2^i$ for $i = 0,1,2,...,s-1$.

DoG Pyramid

- Since DoG calculation $ff(x, y, \sigma_1) - ff(x, y, \sigma_2)$ requires that the (x, y) coordinates at the point at which the difference is being computed be the same in the two Gaussian-smoothed images. We do not change the image resolution in the within-octave samplings of the scale space. Therefore, the scale-space would now look like as shown at left of the DoG Pyramid figure shown before (also shown in the next slide).
- If we want to use the same digital Gaussian smoothing operator at the octave boundaries that you do at the within-octave levels, you must:
  - First downsample the image in proportion to the scale factor,
  - Apply the operator, and, finally
  - Upsample the result so that its resolution is restored.
- The downsampling (and upsampling) will involve calculating image gray levels at fractional coordinates — something that can easily be accomplished with, say, bilinear interpolation.

DoG Pyramid

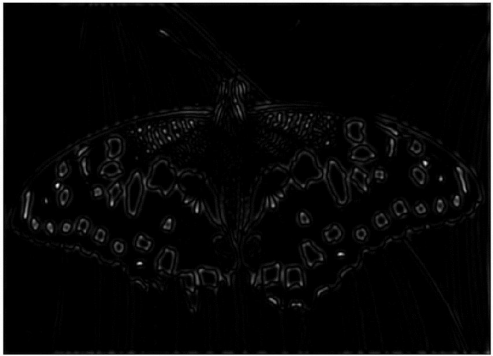
- As shown at right in the figure below, subtracting the adjacent Gaussian-smoothed images within the same ocatve gives us the laplacian at the scale that corresponds to the Gaussian-smoothed images.



Example1 - Assume given the below image

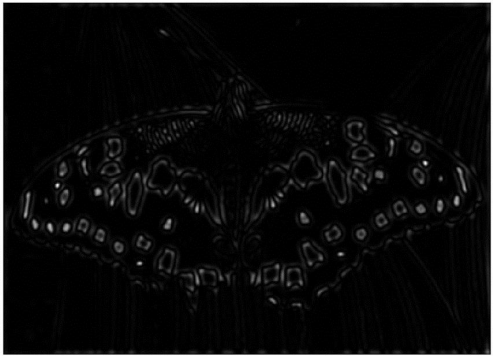


Example1: Applying Original LoG Operator - $\sigma = 2$



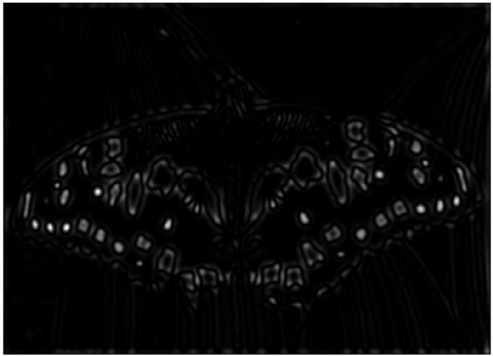
sigma = 2

Example1: Applying Original LoG Operator - $\sigma = 2.5018$



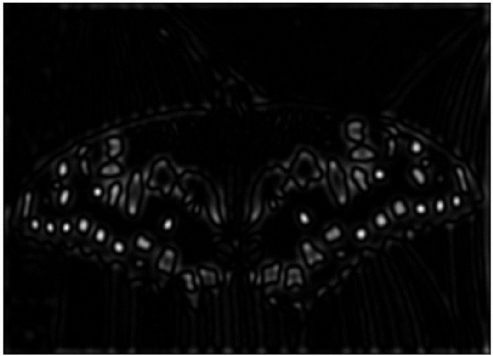
sigma = 2.5018

Example1: Applying Original LoG Operator - $\sigma = 3.1296$



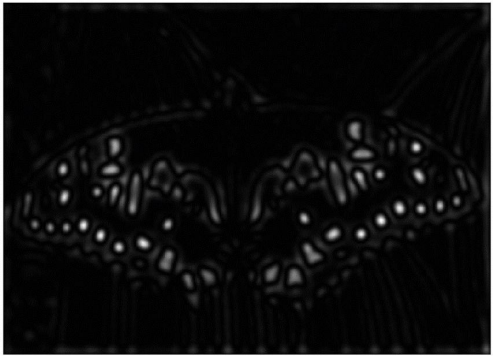
sigma = 3.1296

Example1: Applying Original LoG Operator - $\sigma = 3.9149$



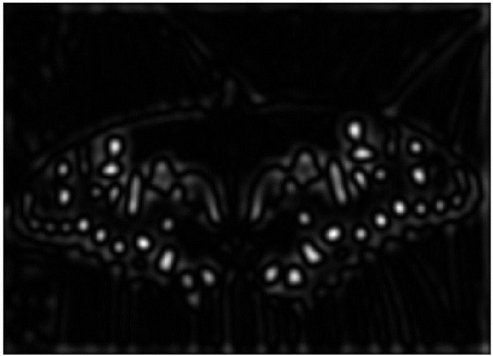
sigma = 3.9149

Example1: Applying Original LoG Operator - $\sigma = 4.8972$



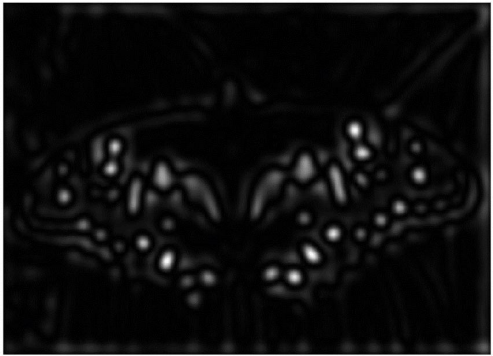
sigma = 4.8972

Example1: Applying Original LoG Operator - $\sigma = 6.126$



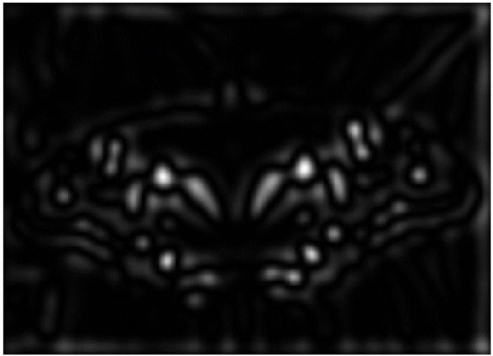
sigma = 6.126

Example1: Applying Original LoG Operator - $\sigma = 7.6631$



sigma = 7.6631

Example1: Applying Original LoG Operator - $\sigma = 9.5859$



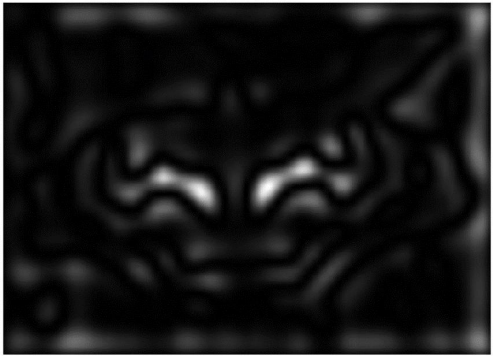
sigma = 9.5859

Example1: Applying Original LoG Operator - $\sigma = 11.9912$



sigma = 11.9912

Example1: Applying Original LoG Operator - $\sigma = 15$



sigma = 15

SIFT Features

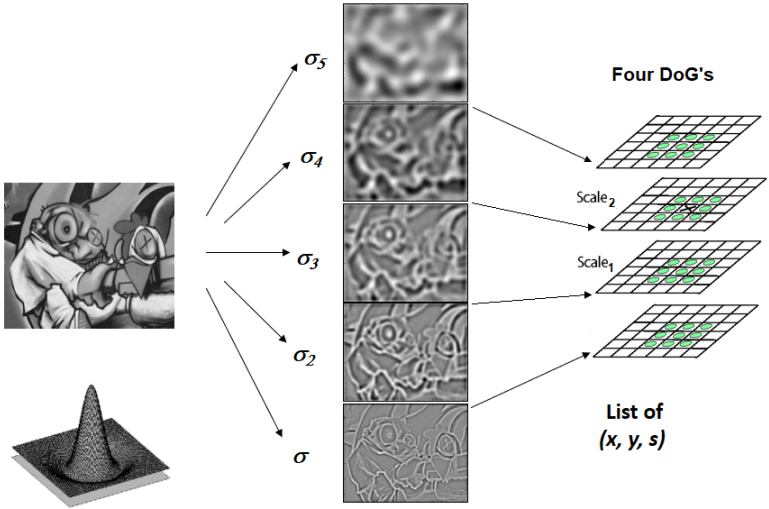
SIFT for Extracting Interest Points and Their Descriptors

- SIFT stands for Scale Invariant Feature Transform.
- Referring to the DoG pyramid, let $D(x, y, \sigma)$ denote the DoG values at scale σ. Shown below are the steps that go into the extraction of the SIFT features.

SIFT: STEP 1

- Find all the local extrema in the DoG pyramid. That is, we find points that are either locally maximum or locally minimum in the three dimensional space (x, y, σ). Each point in the DoG pyramid is compared with:
  - (a) The 8 points in the immediate 3×3 neighborhood at the same scale;
  - (b) The 9 points in the 3×3 neighborhood in the DoG that is at the next level up in the scale space; and
  - (c) The 9 points in the 3×3 neighborhood that is at the level just below.
- Therefore, for a point in the pyramid to pass the local extrema test, it must be compared with 26 neighbors in a $3 \times 3 \times 3$ "volumetric" neighborhood.

SIFT: STEP 1(Cont...)



Aside: An extrema in an image

- In image processing, extrema refer to the points of maximum or minimum intensity within an image.
  - **Maximum:** A pixel where the intensity is higher than all its neighboring pixels.
  - **Minimum:** A pixel where the intensity is lower than all its neighboring pixels.

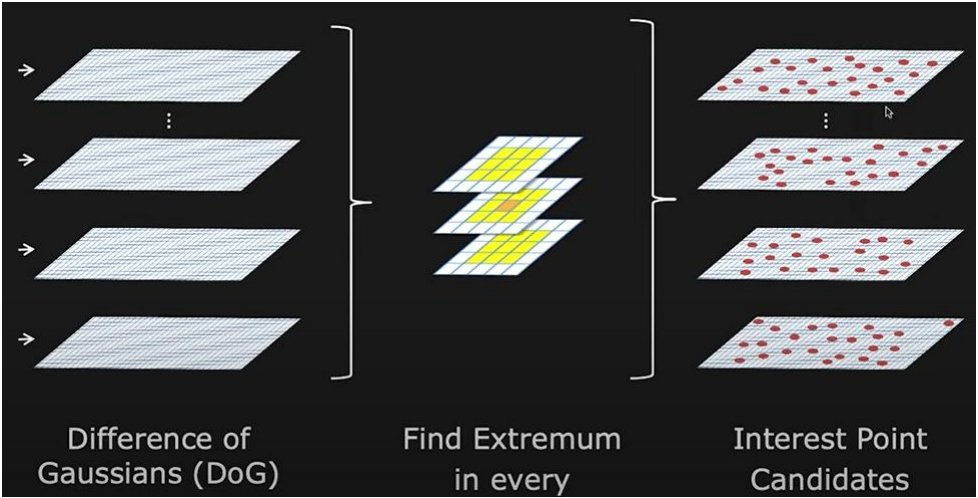
SIFT: STEP 1 (Cont...)

- Obviously, $D(x, y, \sigma)$ must be known for at least 3 values of σ in an octave for the extrema to be identified.
  - It is recommended that you find the extrema for at least two different values of scale σ in each octave.
  - For that we need $D(x, y, \sigma)$ for at least 4 values of σ to compute two exterma's in each octave.
    - That in turn implies that we need Gaussian smoothed images at at-least five different scales in each octave.

SIFT: STEP 1 (Cont...)

- A good question to ask at this point is: **What gray level distributions in the original image give rise to extrema in the DoG pyramid?**
  - For a general answer, points in the original image where the gray levels change rapidly in several directions are likely to give rise to DoG extrema.
  - For a more specific answer, consider an image whose gray levels are all 0 except for a circle of diameter 32 pixels where the gray levels all 1. Now consider convolving this image with the LoG operator at scale σ. As you will recall, the central lobe of this operator is of diameter $2\sqrt{2}\sigma$. So for the scale value $\sigma = 32/2\sqrt{2}$, the central part of the operator will match the size of the circle and the result of the convolution will be the largest value as the operator is right on top of the circle part of the image.

SIFT: STEP 2


Difference of
Gaussians (DoG)
Find Extremum
in every
Interest Point
Candidates

SIFT: STEP 2 - Self-Read

- As σ increases, and especially when you go from a lower octave to a higher octave, the individual discrete points of the DoG represent increasingly coarse sampling of the original image.
- An extremum located at a scale that is coarse may not point directly to a pixel in the original image that should represent a SIFT point.
- We can achieve greater precision in the localization of the extremum vis-a-vis the original image if we improve the location of the extremum in the DoG pyramid.
- As it turns out, if you can estimate the second-order derivatives of D at the sampling points in the DoG pyramid, you can localize the extremum with "sub-pixel" accuracy in the vicinity of where the extremum was found in STEP 1.

SIFT: STEP 2 (Cont...)- Self-Read

- This is how you do it: The Taylor series expansion (for a scalar function of multiple variables) of $D(x, y, \sigma)$ in the vicinity of $\vec{x_0} = (x_0, y_0, \sigma_0)^T$, which was found to be the location of an extremum in STEP 1, can be expressed as:
  - $D(\vec{x}) \simeq D(\vec{x_0}) + J^T(\vec{x_0})\vec{x} + \frac{1}{2}\vec{x}^T H(\vec{x_0})\vec{x}$
  - Where $\vec{x}$ is the incremental deviation from $\vec{x_0}$, and where J is the gradient vector estimated at $\vec{x_0}$: $J(\vec{x_0}) = \left(\frac{\partial D}{\partial x}, \frac{\partial D}{\partial y}, \frac{\partial D}{\partial \sigma}\right)^T_{\vec{x_0}}$, and where H is the Hessian at $\vec{x_0}$:

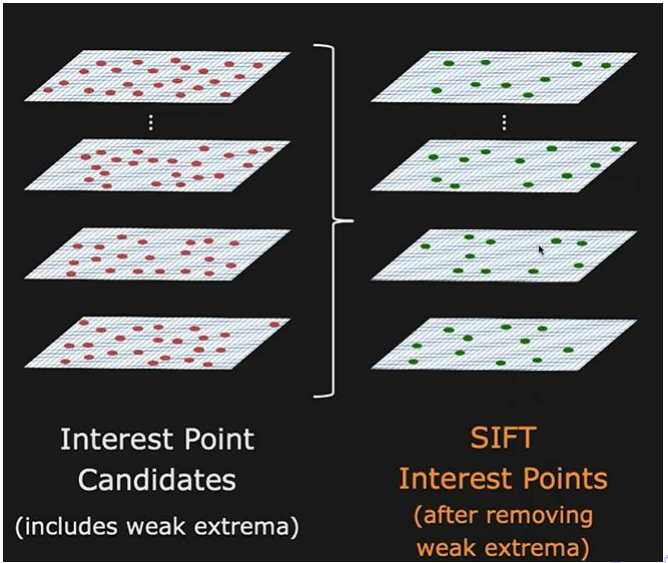
$$H(\vec{x_0}) = \begin{bmatrix} \frac{\partial^2 D}{\partial x^2} & \frac{\partial^2 D}{\partial x \partial y} & \frac{\partial^2 D}{\partial x \partial \sigma} \\ \frac{\partial^2 D}{\partial y \partial x} & \frac{\partial^2 D}{\partial y^2} & \frac{\partial^2 D}{\partial y \partial \sigma} \\ \frac{\partial^2 D}{\partial \sigma \partial x} & \frac{\partial^2 D}{\partial \sigma \partial y} & \frac{\partial^2 D}{\partial \sigma^2} \end{bmatrix}_{at\ \vec{x_0}}$$

  - At the true extremum it must be the case that $\frac{\partial D(\vec{x})}{\partial \vec{x}} = 0$.
  - Taking a derivative of both sides of the Taylor series expansion with respect to the vector variable $\vec{x}$, we get $0 \simeq J(\vec{x_0}) + H(\vec{x_0})\vec{x}$. This implies that the true location of the extremum is given by $\vec{x} = -H^{-1}(\vec{x_0}).J(\vec{x_0})$

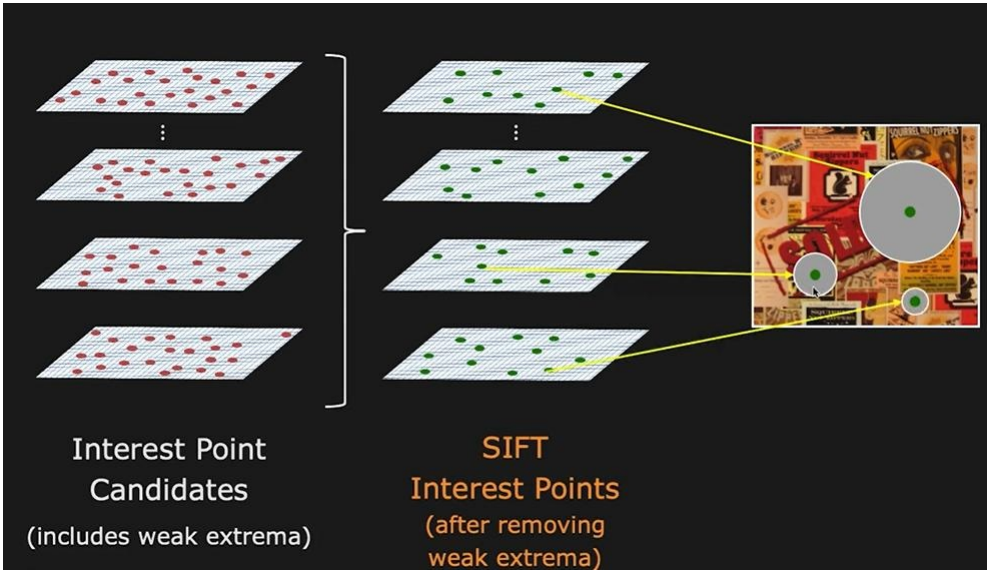
SIFT: STEP 3

- Weed out the weak extrema by thresholding $|D(\vec{x})|$ at the locations $(x, y, \sigma)^T = \vec{x}$ of the extremums. Typically, an extremum is rejected if $|D(\vec{x})| < 0.03$. We also reject those extrema in the scale space that draw their support from an edge in the image. (Edges parallel to the sampling axes are unlikely to give rise to DoG pyramid extrema on account of the logic in STEP 1, but slanted edges might.) This elimination of extrema uses the same logic that you will see later for the Harris corner detector. The Harris corner detection logic is applied at the same scale at which the extremum was discovered.

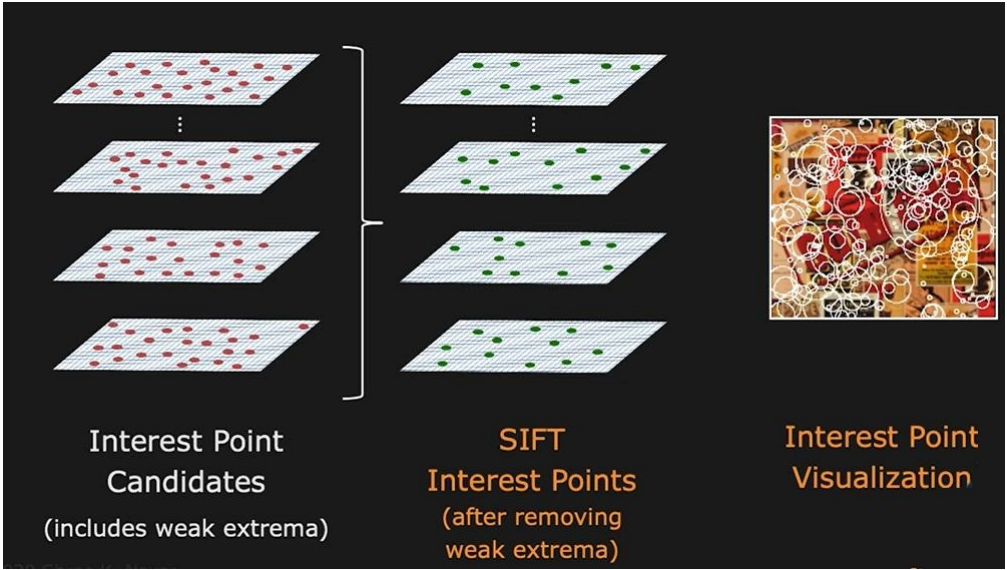
SIFT: STEP 3 - Thresholding Scheme



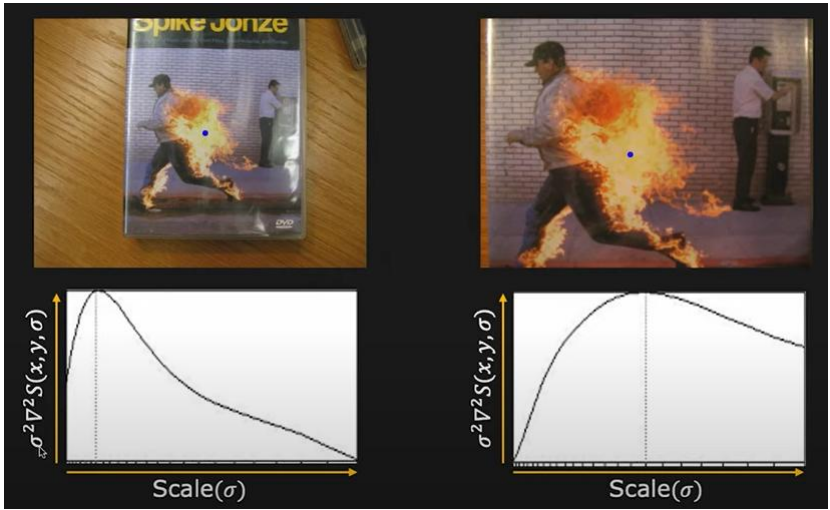
SIFT: STEP 3


Interest Point
Candidates
(includes weak extrema)
SIFT
Interest Points
(after removing
weak extrema)

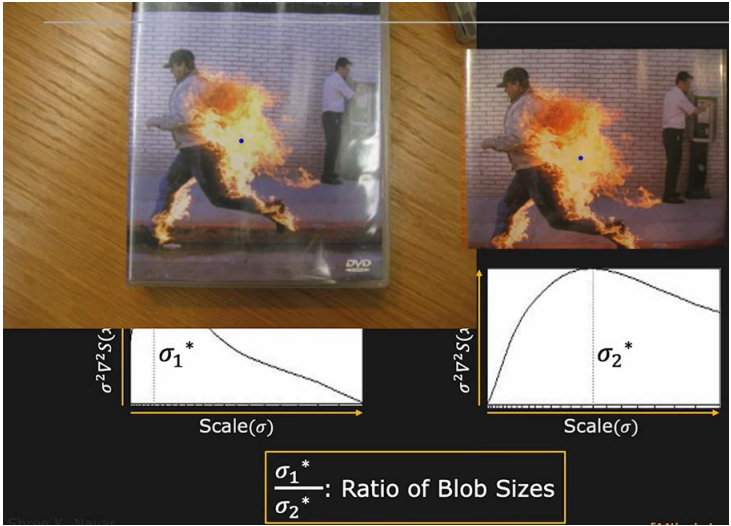
SIFT: STEP 3


Interest Point Candidates
(includes weak extrema)
SIFT Interest Points
(after removing weak extrema)
Interest Point Visualization

SIFT Scale Invariance: Normalize based on ratio of blob sizes $\frac{\sigma_1}{\sigma_2}$

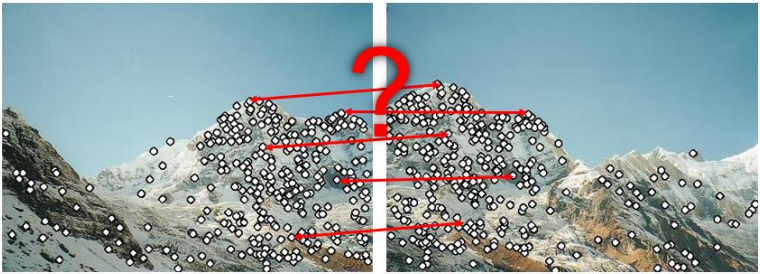


SIFT Scale Invariance: Rescale up



Sift Descriptor

We know how to detect good points
Next question: **How to match them?**



Answer: Come up with a *descriptor* for each point, find similar descriptors between the two images

SIFT: STEP 4

- We now associate a **"dominant local orientation"** or **Principal Orientation** with each extrema that survives the previous step. Note that our ultimate goal is to associate a "descriptor" with each retained extremum.
- To make these descriptors invariant to in-plane rotations of the recorded image, we must calculate them with respect to the dominant local orientation.

SIFT: STEP 4 (Cont...)

- To find the dominant local orientation, we calculate the gradient vector of the Gaussian-smoothed image $ff(x,y,\sigma)$ at the scale σ of the extremum. At each point in a $K \times K$ neighborhood around the extremum, we compute the gradient magnitude $m(x,y) = \sqrt{|ff(x+1,y,\sigma) - ff(x,y,\sigma)|^2 + |ff(x,y+1,\sigma) - ff(x,y,\sigma)|^2}$ and the gradient orientation $\theta(x,y) = arctan \frac{ff(x,y+\mathbf{1},\sigma) - ff(x,y,\sigma)}{ff(x+\mathbf{1},\mathbf{1},\sigma) - ff(x,y,\sigma)}$
- After weighting $\theta(x,y)$ with $m(x,y)$, we construct a histogram of $\theta(x,y)$ values using 36 bins spanning the full 360°range.
- The bin where the histogram peak occurs gives us the **dominant local orientation**.
  - For better accuracy, we can fit a parabola to the peak and its immediate neighbors.
- Note: we only care here in the gradient orientation and **ignore gradient magnitude** as it is usually sensitive to light effects or in other words it is affected by the brightness or illumination or the gain of the image. **This makes SIFT features Illumination Invariance.**

Use the histogram of gradient directions

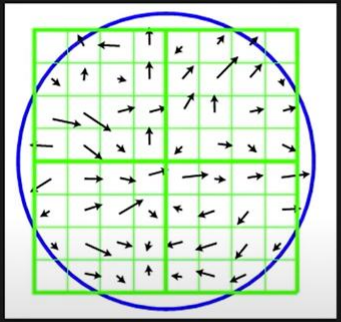
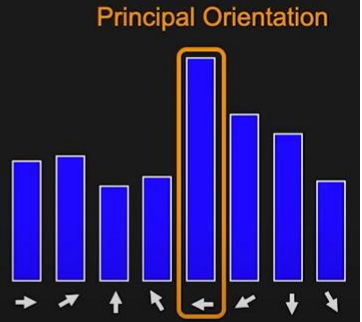


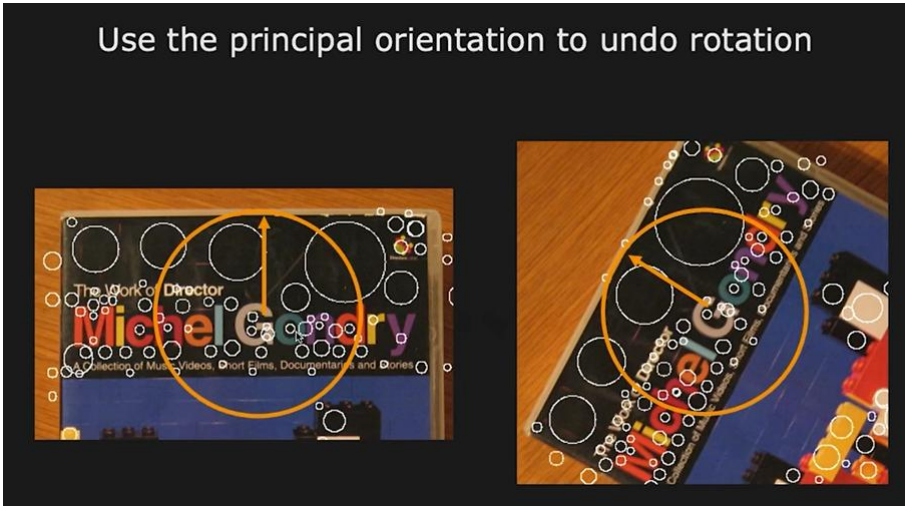
Image gradient directions



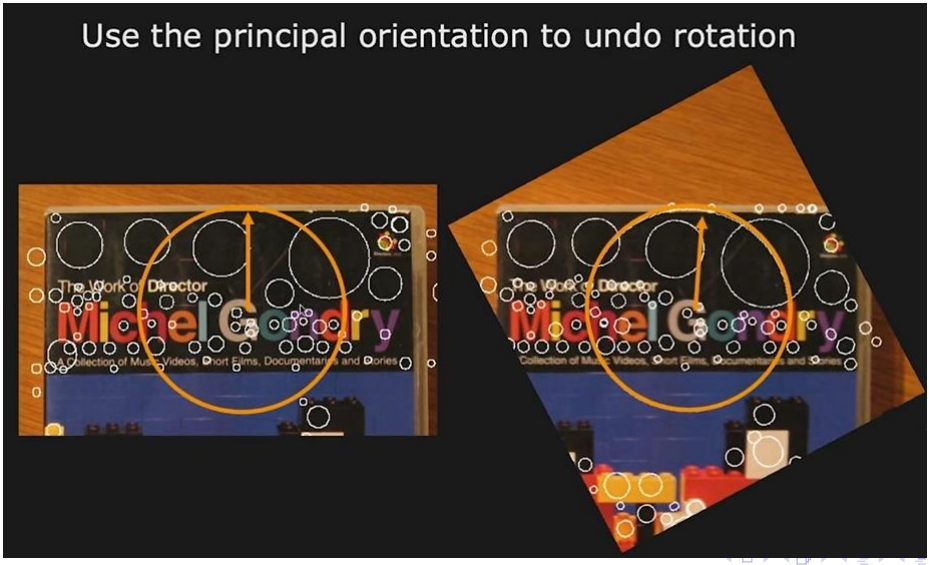
Choose the most prominent gradient direction

$$\theta = \tan^{-1}\left(\frac{\partial I}{\partial y} / \frac{\partial I}{\partial x}\right)$$

SIFT Rotation Invariance



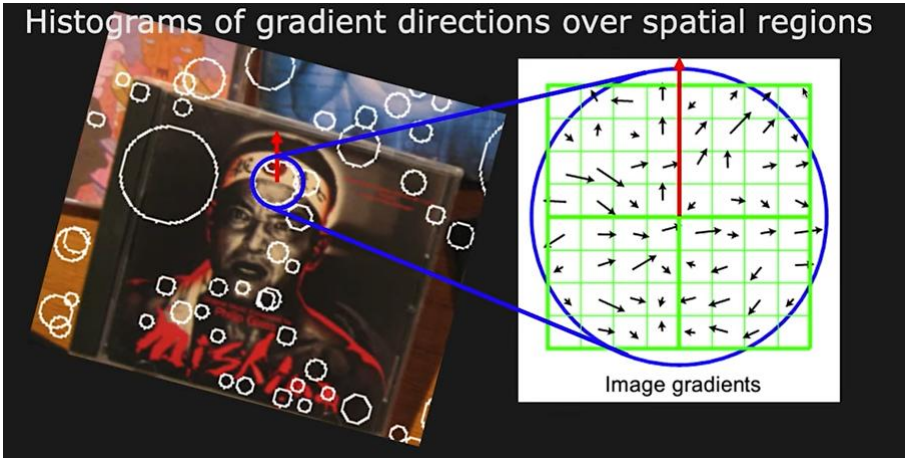
SIFT Rotation Invariance



SIFT: STEP 5 - Describing Each Feature (Signature)

- This is where we create the SIFT descriptor — a 128-dimensional vector — at each retained extremum in the DoG pyramid. We use the same gradient information as was calculated in the previous step, except that the gradient directions $\theta(x, y)$ are now measured relative to the dominant local orientation.
- At the scale of the extremum, the extremum point is surrounded by a 16×16 neighborhood of points that is divided into 4×4 cells, each cell consisting of a 4×4 block of points.
- The magnitudes of the gradients in the 16×16 neighborhood are weighted by a Gaussian, where σ is half the width of the neighborhood in order to reduce the importance of the points that are relatively far from the extremum.

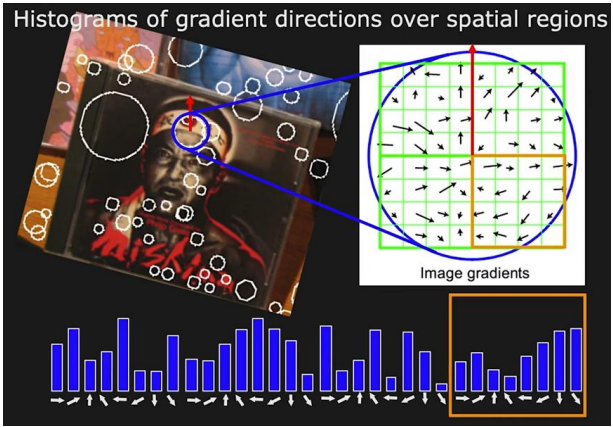
SIFT: STEP 5 (Cont...)



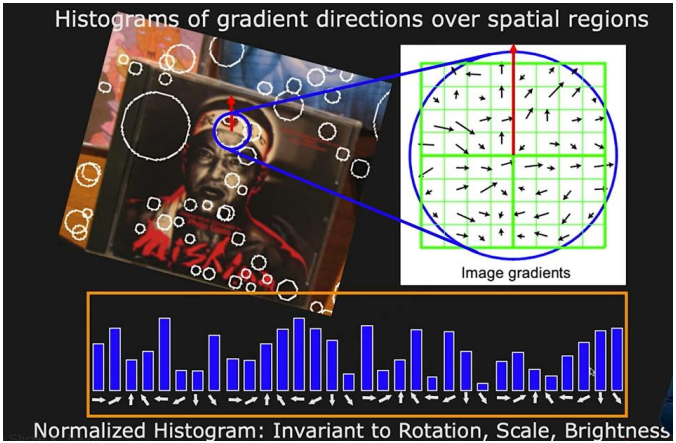
SIFT: STEP 5 (Cont...)

- For each of the 16 cells, an 8-bin orientation histogram is calculated from the gradient-magnitude-weighted values of $\theta(x, y)$ at the 16 pixels in the cell.
- Stringing together the 16 8-bin histograms yields a 128-element descriptor at each retained extremum in the DoG pyramid.
- Considering the 128-element descriptor as a vector in a 128-dimensional space, its length is normalized to unity to make it invariant to changes in illumination.

SIFT: STEP 5 (Cont...) - Normalized Histogram

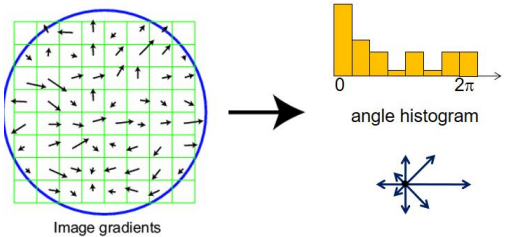


SIFT: STEP 5 (Cont...) - Normalized Histogram



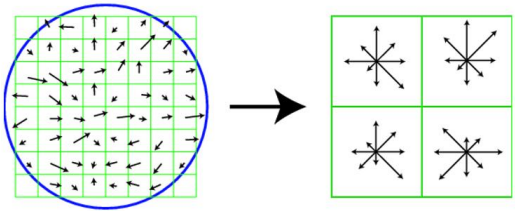
SIFT: Summary

- Basic idea:
  - Take 16x16 square window around detected feature Compute edge orientation (angle of the gradient - 90) for each pixel
  - Throw out weak edges (threshold gradient magnitude)
  - Create histogram of surviving edge orientations
  - Shift the bins so that the biggest one is first

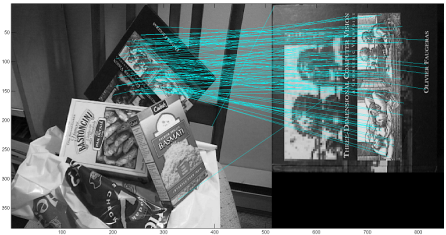


SIFT: Summary

- SIFT descriptor
  - Divide the 16x16 window into a 4x4 grid of cells (2x2 case shown below)
  - Compute an orientation histogram for each cell
  - 16 cells * 8 orientations = 128 dimensional descriptor



SIFT: Demo



- Please visit following link: `https://www.cs.ubc.ca/~lowe/keypoints/`
  - Or you can directly download the SIFT demo at the following link: `http://www.cs.ubc.ca/~lowe/keypoints/siftDemoV4.zip`
- Additionally, you can look at the following link: `https://docs.opencv.org/4.x/da/df5/tutorial_py_sift_intro.html`

References

- "Computer Vision: Algorithms and Applications" by Szeliski (2nd edition).
- "Multiple View Geometry in Computer Vision" by Hartley and Zisserman.
- "First Principles of Computer Vision"
- Prof. Avi Kak's Computer Vision Lectures at Purdue University.